[CYBERTEC] blog - Performance: https://www.cybertec-postgresql.com/en/category/performance/
[pganalyze] blog: https://pganalyze.com/blog

==========

==Subquery/CTE==

CTE: https://www.postgresql.org/docs/current/queries-with.html

CTE
MATERIALIZE
A CTE may be **MATERIALIZED** or **NOT MATERIALIZED**

	MATERIALIZED	**NOT MATERIALIZED**
default	otherwise	when a) **side-effect free i.e.** no **volatile** function calls NOTE: **aggregates** and **window** functions are immutable b) **non-recursive** c) used only **once**
merged into parent	NO	YES
parent restriction e.g. parent WHERE	NOT propagated	propagated
can use index i.e. if no query always CTE SCAN	NO	YES

e.g. parent **restriction propagation**
Parent restrictions propagated

```
WITH cte AS (                       -- OK
    SELECT id, group, value
    FROM b                          -- same as WHERE b.group = '1'
  )
  SELECT *
  FROM a
  INNER JOIN cte ON
    a.id        = cte.id
    AND a.group = cte.group
  WHERE
    a.group = '1';
```

Parent restrictions NOT propagated

```
WITH cte AS MATERIALIZED (        -- BAD. cte materialized all of
b
    SELECT id, group
    FROM b
  )
  SELECT *
  FROM a
  INNER JOIN cte ON
    a.id        = cte.id
    AND a.group = cte.group
  WHERE
    a.group = '1';


WITH cte AS MATERIALIZED (        -- OK. cte materializes only b
with group='1'
    SELECT id, group
    FROM b
    WHERE
     b.group = '1';
  )
  SELECT *
  FROM a
  INNER JOIN cte ON
    a.id        = cte.id
    AND a.group = cte.group
  WHERE
    a.group = '1';
```

*** CTE performance: Index Scan with CTE**

Index Scan is possible only if:

a) **MATERIALIZATION**: NOT MATERIALIZED only
i.e. **CTE SCAN** only for MATERIALIZED ctes.

b) parent **restriction propagation** of the index filter
i.e. the **index filter** needs to be propagated **inside** the CTE.

e.g. Materialization
a) without **MATERIALIZE**

```
WITH cte AS (
  SELECT * FROM b
)
SELECT *
FROM a
INNER JOIN cte ON a.id = cte.id


SELECT *                          -- same i.e. merged into parent
FROM a
```

```
INNER JOIN b ON a.id = b.id
```

Query plan

```
Nested Loop
  -> Seq Scan on a
  -> Index Scan on b
     Filter: a.id = b.id
```

b) with **MATERIALIZE**

```
WITH cte AS MATERIALIZE (
  SELECT * FROM b
)
SELECT *
FROM a
INNER JOIN cte ON a.id = cte.id
```

Query plan
CTE scan only.
Can be very slow if the CTE SCAN is not the inner
e.g.
1) outer=**CTE scan** inner=**index** scan OK
2) outer=seq scan inner=CTE scan Might be very slow

```
-- Query plan 1: OK
Nested Loop
  -> CTE Scan on cte
  -> Index Scan on a                    -- Fast
     Filter: a.id = cte.id

CTE cte
  -> Seq Scan on b


-- Query plan 2: Slow
Nested Loop
  Join condition: a.id = cte.id
  -> Seq Scan on c
  -> CTE Scan on cte

CTE cte
  -> Seq Scan on b
```

** **GOTCHA: CTE columns** does not match **index** columns
MUST use **same index** as the original table.

e.g. if index of tables is (id, group)

```
WITH cte AS (                       -- GOOD. Uses index b (id,
group)
    SELECT
      b.id, b.group
    FROM b
    INNER JOIN c ON
      b.id        = c.id
      AND b.group = c.group
  )
SELECT *
FROM a
INNER JOIN cte ON                   -- OK
  a.id        = cte.id              -- a.id    = b.id
  AND a.group = cte.group           -- a.group = b.group


WITH cte AS (                       -- BAD: Mix of b and c
    SELECT
      b.id, c.group
    FROM b
    INNER JOIN c ON
      b.id        = c.id
      AND b.group = c.group
  )
SELECT *
FROM a
INNER JOIN cte ON                   -- NO
  a.id        = cte.id              -- a.id    = c.id
  AND a.group = cte.group           -- a.group = c.group
```

*** [GOTCHA] CTE performance: slow MATERIALIZED CTE in JOIN**

MATERIALIZED CTEs do not have indexes.
i.e. sequential **CTE SCAN** only.
Hence, the query may be very slow in **JOINs**

e.g. slow Nested Loop

```
WITH cte AS MATERIALIZE (
  SELECT * FROM b
)
SELECT *
FROM a
INNER JOIN cte ON a.id = cte.id
```

Query plans
1) outer=**CTE scan** inner=**index** scan OK
2) outer=seq scan inner=CTE scan Might be very slow

```
-- Query plan 1: OK
Nested Loop
  -> CTE Scan on cte
  -> Index Scan on a                    -- Fast
     Filter: a.id = cte.id

CTE cte
  -> Seq Scan on b


-- Query plan 2: Slow
Nested Loop
  Join condition: a.id = cte.id
  -> Seq Scan on c
  -> CTE Scan on cte

CTE cte
  -> Seq Scan on b
```

*** [GOTCHA] aggregate/window function performance - aggregation in a Nested Loop JOIN**

Problem:
By default, the **aggregation** is **computed every time** the row is evaluated.
This can be very expensive depending on the query plan.
In particular, if **Nested Loop JOIN** + aggregate in **inner table**.

e.g. window function in a subquery/CTE

```
-- window function in CTE
WITH c AS (
   SELECT
      parent_id,
      value,
      RANK() OVER (
         PARTITION BY parent_id, child_id
         ORDER BY value
      ) rank
   FROM child
)
SELECT ...
FROM parent
INNER JOIN c ON
  parent.id = c.parent_id
WHERE
  c.rank = 1


-- analogous in subqeury
SELECT ...
FROM parent
INNER JOIN (
```

```
  SELECT
      parent_id,
      value,
      RANK() OVER (
         PARTITION BY parent_id, child_id
         ORDER BY value
      ) rank
   FROM child
) c ON
  parent.id = c.parent_id
WHERE
  c.rank = 1
```

Potential query plans

a) aggregate in **outer** loop	OK
b) aggregate in **inner** loop with **Index** Scan	OK
c) aggregate in **inner** loop with **Seq** Scan	BAD.

```
Nested Loop                                                 a) GOOD
  Join condition: parent.id = c.parent_id
  -> Subquery Scan on c                                     Loop=1 of c
     Filter: c.rank = 1
     -> Window Agg
        -> Sort
           Sort Key: parent_id, child_id, value
           -> Seq Scan on child
  -> Index Scan on parent
     Filter: parent_id = c.parent_id

Nested Loop                                                 b) OK
  Join condition: parent.id = c.parent_id
  -> Seq Scan on parent
  -> Subquery Scan on c
     Filter: c.rank = 1
     -> Window Agg
        -> Sort
           Sort Key: parent_id, child_id, value
           -> Index Scan on child                           aggregate
only with matching rows of child
               Filter: parent.id = child.parent_id


Nested Loop                                                 c) BAD
  Join condition: parent.id = c.parent_id
  -> Seq Scan on parent
  -> Subquery Scan on c
     Filter: c.rank = 1
     -> Window Agg                                          SLOW: all
aggregations recomputed for each row of parent
        -> Sort                                             i.e. (#
parents) * (# child) loops
```

```
            Sort Key: parent_id, child_id, value
            -> Seq Scan on child
```

Solution 1 (not great): **MATERIALIZE** the aggregation
Major drawback: no **index** of the materialized table.
e.g. window function in a subquery/CTE

```
-- window function in CTE
WITH c AS MATERIALIZED (
   SELECT
      parent_id,
      value,
      RANK() OVER (
         PARTITION BY parent_id, child_id
         ORDER BY value
      ) rank
   FROM child
)
SELECT ...
FROM parent
INNER JOIN c ON
  parent.id = c.parent_id
WHERE
  c.rank = 1
```

Improved query plan with CTE in **inner table**

```
Nested Loop
  Join condition: parent.id = c.parent_id
  -> Seq Scan on parent
  -> CTE Scan on c                                Still (# parents) *
(# child) loops
                                                  but aggregation is
only computed once

CTE c
  -> Window Agg
     -> Sort
        Sort Key: parent_id, child_id, value
        -> Seq Scan on child
```

Solution 2 (best): aggregate in a separate table/Materialized view
e.g. materialized view with appropriate indexes

```
CREATE MATERIALIZED VIEW child_with_rank AS
   SELECT
      parent_id,
```

```
      value,
      RANK() OVER (
         PARTITION BY parent_id, child_id
         ORDER BY value
      ) rank
   FROM child;

CREATE INDEX ON child_with_rank (parent_id, rank);


-- Query
SELECT ...
FROM parent
INNER JOIN child_with_rank c ON
  parent.id = c.parent_id
WHERE
  c.rank = 1
```

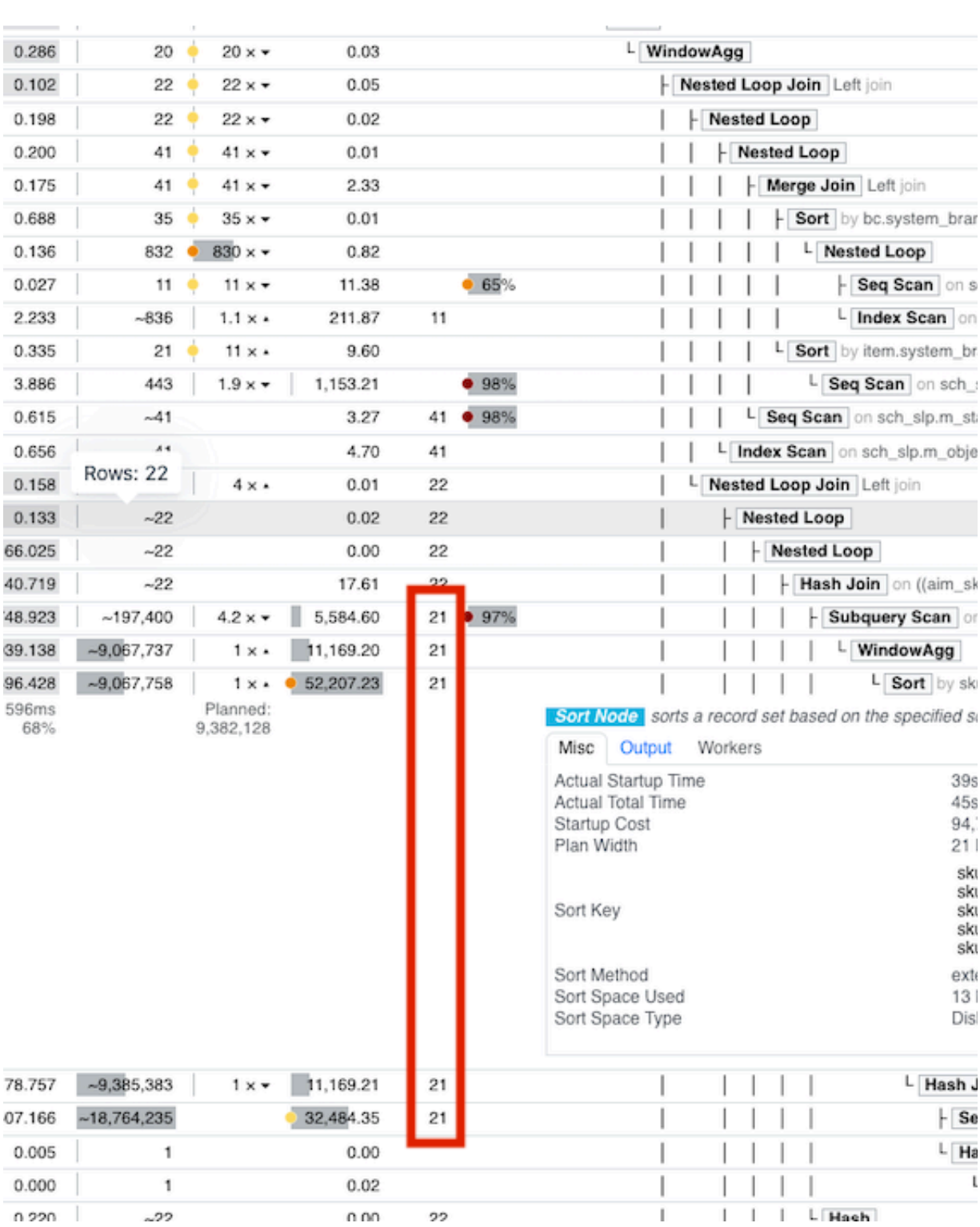
e.g. real example
SQL



EXPLAIN result



Subquery is repeated 21 times.



==========

==Memory and buffers==

[The internals of Postgres] 2.1 Memory architecture: https://www.interdb.jp/pg/pgsql02/02.html
[The internals of Postgres] 3.4 Executor performance: https://www.interdb.jp/pg/pgsql03/04.html
[The internals of Postgres] 8. Buffer Manager: https://www.interdb.jp/pg/pgsql08/index.html

Buffer (aka cache)
14.1 Using Explain: https://www.postgresql.org/docs/current/using-explain.html#USING-

EXPLAIN-ANALYZE
[BoringSQL] Introduction to buffers: https://boringsql.com/posts/introduction-to-buffers/
[BoringSQL] Reading buffer stats: https://boringsql.com/posts/explain-buffers/
Postgres **pages** i.e. data are permanently stored as **files** on disk.
Hence, by default, **page access** is a **OS file read**, which can be slow.
Buffers are in-**memory cache** of pages for faster access (EXCEPT for **temp buffer**=swap file)
Buffers are stored in units of **pages**.

3 types of buffers:
1) **[memory] shared buffer**: **memory cache** for DB data
i.e. faster access to data through **memory** than **file read**.
Shared buffer is stored in **shared memory**.
Buffers are stored in units of **pages**.
Default is **128MB**.

e.g.
hit = cache hit
read = cache miss

```
>slpadmin@localhost:5432/slp[state=]# EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM pg_stats;

QUERY PLAN
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────
──────────────────────────
 Nested Loop Left Join  (cost=1026.79..3411.78 rows=14 width=401)
(actual time=4.681..27.980 rows=701 loops=1)
   Buffers: shared hit=7469 read=3 dirtied=7
   ->  Nested Loop  (cost=1026.64..3409.56 rows=14 width=475)
(actual time=4.671..27.127 rows=701 loops=1)
         Join Filter: (s.staattnum = a.attnum)
         Rows Removed by Join Filter: 9488
         Buffers: shared hit=7463 read=3 dirtied=7
...

 Planning:
   Buffers: shared hit=149 read=1
 Planning Time: 3.267 ms
 Execution Time: 28.418 ms
(34 rows)
```

2) **[memory] local buffer**: per-**client** buffer.
Buffer of a **temporary table**.

e.g.

```
> CREATE TEMPORARY TABLE mytable AS (SELECT * FROM pg_tables);
SELECT 2436

> EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM pg_temp.mytable;
                                                    QUERY PLAN
```

```
_____________________________________________________________________
_____________________________________
 Seq Scan on mytable  (cost=0.00..89.60 rows=1960 width=260)
(actual time=0.036..1.431 rows=2436 loops=1)
   Buffers: local hit=70
 Planning:
   Buffers: shared hit=18
 Planning Time: 0.481 ms
 Execution Time: 1.649 ms
```

3) **[file] temp (temporary file)**: like **swap** for **work_mem**
i.e. if a query operation is too big to fit in work_mem, the memory is written to a **temporary file**.

e.g.
Temp file used by Sort

```
> SET work_mem TO '256kB';

> EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM pg_tables ORDER BY
tablename;

                                                          QUERY
PLAN
_____________________________________________________________________
_____________________________________________________________
 Sort  (cost=1398.91..1405.00 rows=2435 width=260) (actual
time=18.151..18.769 rows=2436 loops=1)
   Sort Key: c.relname
   Sort Method: external merge  Disk: 504kB
   Buffers: shared hit=818, temp read=63 written=66
   ->  Hash Left Join  (cost=2.14..960.94 rows=2435 width=260)
(actual time=0.958..7.396 rows=2436 loops=1)
         Hash Cond: (c.reltablespace = t.oid)
...
```

*** Note: Temporary file (temp file) vs temp buffer vs swap**
[The internals of Postgres] 3.4.1.2 Temporary files: https://www.interdb.jp/pg/pgsql03/04.html
how to debug spilling to disk in AWS RDS: https://nataliiadziubenko.com/2025/07/17/aws-rds-how-to-debug-spilling.html

Similar functionality/name but are not the same.

	System	Storage	
temporary file (aka temp file)	Postgres	disk	like swap for PG i.e. **spill** to disk when query uses more than **work_mem**.

temp buffer for **TEMPORARY tables**	Postgres	memory	like **shared_buffer**
swap	OS	memory/disk	OS

*** Configurations**

Resource configs: https://www.postgresql.org/docs/current/runtime-config-resource.html

Note: **system only** configs can only be set in a **server config file (postgresql.conf)** or **server CLI**.

Config	Level	Default
shared_buffer Total **shared memory** i.e. **page** cache size	system only	128MB
work_mem **Base** memory used by a **query node**. **Actual** memory may be different e.g. Sort work_mem Materialize work_mem Memoize hash_multiplier * work_mem Hash hash_multiplier * work_mem NOTE: **each** query node, NOT the whole query i.e. whole query may use **# nodes * work_mem** e.g. if work_mem=10MB, Merge Sort -> Sort max 10MB ... -> Sort max 10MB ... ->		4MB
temp_buffers Memory for **TEMPORARY** tables.		8MB

maintenance_work_mem		64MB
e.g. VACUUM, CREATE INDEX,		
autovacuum_work_mem	system only	-1
Memory used by **AUTOVACUUM** (not VACUUM)		
-1 = same amount as maintenance_work_mem		
logical_decoding_work_mem		64MB
transaction_buffers	system only	
subtransaction_buffers	system only	
serializable_buffers	system only	
notify_buffers	system only	
multixact_offset_buffers	system only	
multixact_member_buffers	system only	
commit_timestamp_buffers	system only	
logical_decoding_work_mem	system only	

==========

== INDEX==

CREATE INDEX

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ [ IF NOT EXISTS ] name ]
ON [ ONLY ] table_name [ USING method ]
    ( { column_name | ( expression ) } [ COLLATE collation ]
[ opclass [ ( opclass_parameter = value [, ... ] ) ] ]
    [ ASC | DESC ]                                          --
default: ASC
    [ NULLS { FIRST | LAST } ] [, ...] )                    --
sorting: NULLS FIRST   NULL < NON-NULL

NULLS LAST    NULL > NON-NULL
    [ INCLUDE ( column_name [, ...] ) ]
    [ NULLS [ NOT ] DISTINCT ]                              --
unique key: whether to allow multiple NULLs
    [ WITH ( storage_parameter [= value] [, ... ] ) ]
    [ TABLESPACE tablespace_name ]
    [ WHERE predicate ]
```

ALTER INDEX

```
ALTER INDEX [ IF EXISTS ] name RENAME TO new_name
ALTER INDEX [ IF EXISTS ] name SET TABLESPACE tablespace_name
ALTER INDEX name ATTACH PARTITION index_name
ALTER INDEX name [ NO ] DEPENDS ON EXTENSION extension_name
ALTER INDEX [ IF EXISTS ] name SET ( storage_parameter [= value]
[, ... ] )
ALTER INDEX [ IF EXISTS ] name RESET ( storage_parameter [, ... ] )
ALTER INDEX [ IF EXISTS ] name ALTER [ COLUMN ] column_number
    SET STATISTICS integer
```

```
ALTER INDEX ALL IN TABLESPACE name [ OWNED BY role_name [, ... ] ]
    SET TABLESPACE new_tablespace [ NOWAIT ]
```

DROP INDEX

```
DROP INDEX [ CONCURRENTLY ] [ IF EXISTS ] name [, ...] [ CASCADE |
RESTRICT ]
```

11. Indexes: https://www.postgresql.org/docs/current/indexes.html
36.16. Interfacing extensions: https://www.postgresql.org/docs/current/xindex.html
64. Builtin indexes: https://www.postgresql.org/docs/current/indextypes.html

Index methods
11. Indexes: https://www.postgresql.org/docs/current/indexes-types.html
64. Builtin indexes: https://www.postgresql.org/docs/current/indextypes.html

An **index method aka type** represents a **data structure**/algorithm used to implement indexing.
e.g.

btree index	Btree
hash	hash table

Default index is **btree**.

Operator family
Operator class
11. Indexes: https://www.postgresql.org/docs/current/indexes.html
36.16. Interfacing extensions: https://www.postgresql.org/docs/current/xindex.html
Operator family is a set of **operators** that can be used for indexing
Operator class is an implementation of the **operator family** for one or more concrete types.
Whether an operator can be supported by an **index type** depends on the underlying **data structure** of the index
e.g.

index type	**operator family**	**operator class**	**data type**	**operators**
btree	integer_ops	int2_ops	smallint	=, <, >, ...
btree	integer_ops	int4_ops	integer	=, <, >, ...
btree	numeric_ops	numeric_ops	float, real, etc	=, <, >, ...
btree	text_ops	text_ops	text	=
btree	text_pattern_ops	text_pattern_ops	text	patterns

```
-- integer operators
> \dAc+ btree int[0-9]*
                                    List of operator classes
  AM   | Input type | Storage type | Operator class | Default? |
Operator family |    Owner
═══════╪════════════╪══════════════╪════════════════╪══════════╪═══
═══════════════╪═══════════
 btree | bigint     |              | int8_ops       | yes      |
integer_ops     | yuinishizawa
 btree | integer    |              | int4_ops       | yes      |
```

```
integer_ops     | yuinishizawa
 btree | smallint   |              | int2_ops       | yes      |
integer_ops     | yuinishizawa
(3 rows)

-- numeric operators
> \dAc btree numeric*
                   List of operator classes
  AM   | Input type | Storage type | Operator class | Default?
═══════╪════════════╪══════════════╪════════════════╪══════════
 btree | numeric    |              | numeric_ops    | yes
(1 row)
```

Index types
Ref: https://www.postgresql.org/docs/current/indexes.html

- **unique** index: index with **UNIQUE** constraint
Ref: https://www.postgresql.org/docs/current/indexes-unique.html

- **ORDER BY** index: index that can be used for sorting

e.g.

```
CREATE INDEX sort ON mytable (id ASC);                        // sort
index on "id" column


> EXPLAIN SELECT * FROM mytable ORDER BY id ASC;              // OK.
Use sorted index so no extra sorting needed.

Index Scan using mytable_id_idx on mytable  (...)

> EXPLAIN SELECT * FROM mytable ORDER BY id DESC;             // OK.
Reverse order.

Index Scan Backward using mytable_id_idx on mytable (...)

> EXPLAIN SELECT * FROM mytable ORDER BY x ASC;               // No
index.

Sort  (cost=208.59..215.33 rows=2696 width=41)
  Sort Key: x
  ->  Seq Scan on mytable  (...)
```

- **multicolumn** indexes: column over multiple columns
Multicolumn index: https://www.postgresql.org/docs/current/indexes-multicolumn.html
Currently supported only for **B-tree, GIST, GIN, BRIN**

B-tree rules:
- **leading equality** columns: index is efficiently used for **consecutive leading** columns.

[UPDATE]: PostgreSQL 18 supports **skip scan**.

- **inequality**: index is NOT used after the first inequality column
e.g.

```
CREATE INDEX ON mytable (x, y, z);

-- Leading
SELECT * FROM mytable
  WHERE x = 1             -- Full. Uses index. Most efficient.
   AND y = 2
   AND z = 3

SELECT * FROM mytable
  WHERE y = 2             -- Non-leading i.e. x not specified
   AND z = 3              -- Might use index, but less efficient

SELECT * FROM mytable
  ON y = 2                -- Single.
                          -- Can use index, but less efficient

-- Inequality
SELECT * FROM mytable
  ON x = 1                -- Index only for (x, y)
  AND y >= 2
  AND z = 3
```

- **cover** indexes: values stores in indexed, but not part of index.
Ref: https://www.postgresql.org/docs/current/indexes-index-only-scans.html
Used for **index only queries** i.e. queries that can be processed with only index values, without reading the actual row.
e.g.

```
CREATE INDEX ON mytable (x) INCLUDING (y, z);

SELECT x, a, b FROM mytable WHERE x > 10;      // NOT index only,
since a, b not in index
SELECT x, y, z FROM mytable WHERE x > 10;      // Index only, since
y, z are included in index.
```

- **expression** indexes: index on a column expression i.e. a **computed value**.
Ref: https://www.postgresql.org/docs/current/indexes-expressional.html
e.g.

```
CREATE INDEX myindex ON mytable (floor(f));

SELECT * FROM mytable WHERE floor(f) = 0;      // e.g. use index
```

- **partial** indexes: index on a subset of column values.
Ref: https://www.postgresql.org/docs/current/indexes-partial.html
e.g.

```
CREATE INDEX ON mytable (x) WHERE x > 0;

SELECT * FROM mytable WHERE x = 10;            // use index
SELECT * FROM mytable WHERE x = -10;           // no index
```

usage: IN vs ANY of array
[SO] IN vs ANY in Postgres: https://stackoverflow.com/questions/34627026/in-vs-any-operator-in-postgresql
[SO] Postgres performance of IN vs ANY: https://pganalyze.com/blog/5mins-postgres-performance-in-vs-any

Both use the index with the **equality operator**.
IN gets converted to **equality** with **ANY**.
e.g. ANY and IN produce the same query plan.

```
# ANY
> EXPLAIN  (analyze, verbose)
SELECT store_name FROM m_store
WHERE store_code = ANY(array['1', '2']);

 Index Scan using m_store_pk on sch_slp.m_store  (cost=0.28..15.81
rows=2 width=374) (actual time=0.044..0.056 rows=2 loops=1)
   Output: store_name
   Index Cond: ((m_store.store_code)::text = ANY ('{1,2}'::text[]))
 Query Identifier: -5544616894470697003
 Planning Time: 0.098 ms
 Execution Time: 0.095 ms
(6 rows)


# IN
> EXPLAIN  (analyze, verbose)
SELECT store_name FROM m_store
WHERE store_code IN ('1', '2');

Index Scan using m_store_pk on sch_slp.m_store  (cost=0.28..15.81
rows=2 width=374) (actual time=0.017..0.023 rows=2 loops=1)
   Output: store_name
   Index Cond: ((m_store.store_code)::text = ANY ('{1,2}'::text[]))
 Query Identifier: -8873486918640173884
 Planning Time: 0.088 ms
 Execution Time: 0.059 ms
(6 rows)
```

*** Methods**

Method: B-tree

64.1 B-tree indexes: https://www.postgresql.org/docs/current/btree.html
The default index.

```
> \dAo+ btree numeric*
                        List of operators of operator families
  AM   | Operator family |      Operator       | Strategy | Purpose
| Sort opfamily
═══════╪═════════════════╪═════════════════════╪══════════╪═══════
═╪═══════════
 btree | numeric_ops     | <(numeric,numeric)  |        1 | search
|
 btree | numeric_ops     | <=(numeric,numeric) |        2 | search
|
 btree | numeric_ops     | =(numeric,numeric)  |        3 | search
|
 btree | numeric_ops     | >=(numeric,numeric) |        4 | search
|
 btree | numeric_ops     | >(numeric,numeric)  |        5 | search
|
(5 rows)

> \dAo+ btree text*
                      List of operators of operator families
  AM   | Operator family  |    Operator     | Strategy | Purpose |
Sort opfamily
═══════╪══════════════════╪═════════════════╪══════════╪═════════╪═
═══════════
 btree | text_ops         | <(name,name)    |        1 | search  |
 btree | text_ops         | <=(name,name)   |        2 | search  |
 btree | text_ops         | =(name,name)    |        3 | search  |
 btree | text_ops         | >=(name,name)   |        4 | search  |
 btree | text_ops         | >(name,name)    |        5 | search  |
 btree | text_ops         | <(text,text)    |        1 | search  |
 btree | text_ops         | <=(text,text)   |        2 | search  |
 btree | text_ops         | =(text,text)    |        3 | search  |
 btree | text_ops         | >=(text,text)   |        4 | search  |
 btree | text_ops         | >(text,text)    |        5 | search  |
 btree | text_ops         | <(name,text)    |        1 | search  |
 btree | text_ops         | <=(name,text)   |        2 | search  |
 btree | text_ops         | =(name,text)    |        3 | search  |
 btree | text_ops         | >=(name,text)   |        4 | search  |
 btree | text_ops         | >(name,text)    |        5 | search  |
 btree | text_ops         | <(text,name)    |        1 | search  |
 btree | text_ops         | <=(text,name)   |        2 | search  |
 btree | text_ops         | =(text,name)    |        3 | search  |
 btree | text_ops         | >=(text,name)   |        4 | search  |
 btree | text_ops         | >(text,name)    |        5 | search  |
 btree | text_pattern_ops | ~<~(text,text)  |        1 | search  |
 btree | text_pattern_ops | ~<=~(text,text) |        2 | search  |
 btree | text_pattern_ops | =(text,text)    |        3 | search  |
 btree | text_pattern_ops | ~>=~(text,text) |        4 | search  |
```

```
 btree | text_pattern_ops | ~>~(text,text)  |        5 | search  |
(25 rows)
```

Method: Hash
Hash indexes: https://www.postgresql.org/docs/current/hash-index.html

```
> \dAo+ hash
                                          List of operators of
operator families
  AM  |  Operator family   |                            Operator
| Strategy | Purpose | Sort opfamily
═══════╪════════════════════╪═══════════════════════════════════════════
═══════════╪═══════════════════╪═══════╪═══════════════
 hash | aclitem_ops        | =(aclitem,aclitem)
|        1 | search  |
 hash | array_ops          | =(anyarray,anyarray)
|        1 | search  |
 hash | bool_ops           | =(boolean,boolean)
|        1 | search  |
 hash | bpchar_ops         | =(character,character)
|        1 | search  |
 hash | bpchar_pattern_ops | =(character,character)
|        1 | search  |
 hash | bytea_ops          | =(bytea,bytea)
|        1 | search  |
 hash | char_ops           | =("char","char")
|        1 | search  |
 hash | cid_ops            | =(cid,cid)
|        1 | search  |
 hash | date_ops           | =(date,date)
|        1 | search  |
```

Method: GiST (Generalized Search Tree)
GIST indexes: https://www.postgresql.org/docs/current/gist.html
Geometric operators
e.g. <<, >>, <->

Method: SP-GiST
SP-GiST indexes: https://www.postgresql.org/docs/current/spgist.html

Method: GIN (Generalized Inverted Index)
GIN indexes: https://www.postgresql.org/docs/current/gin.html
For **composite data types**, such as arrays, JSON
e.g. <@, @>, =, &&

```
> \dAo+ gin
                        List of operators of operator families
 AM  | Operator family |       Operator        | Strategy | Purpose
| Sort opfamily
```

```
 gin | array_ops       | &&(anyarray,anyarray) |        1 | search
 gin | array_ops       | @>(anyarray,anyarray) |        2 | search
 gin | array_ops       | <@(anyarray,anyarray) |        3 | search
 gin | array_ops       | =(anyarray,anyarray)  |        4 | search
 gin | jsonb_ops       | @>(jsonb,jsonb)       |        7 | search
 gin | jsonb_ops       | @?(jsonb,jsonpath)    |       15 | search
 gin | jsonb_ops       | @@(jsonb,jsonpath)    |       16 | search
 gin | jsonb_ops       | ?(jsonb,text)         |        9 | search
 gin | jsonb_ops       | ?|(jsonb,text[])      |       10 | search
 gin | jsonb_ops       | ?&(jsonb,text[])      |       11 | search
 gin | jsonb_path_ops  | @>(jsonb,jsonb)       |        7 | search
 gin | jsonb_path_ops  | @?(jsonb,jsonpath)    |       15 | search
 gin | jsonb_path_ops  | @@(jsonb,jsonpath)    |       16 | search
 gin | tsvector_ops    | @@(tsvector,tsquery)  |        1 | search
 gin | tsvector_ops    | @@@(tsvector,tsquery) |        2 | search
(15 rows)
```

Method: BRIN (Block Range Index)
BRIN indexes: https://www.postgresql.org/docs/current/brin.html

---

==Statistics==

27.2 Statistics system: https://www.postgresql.org/docs/current/monitoring-stats.html
68 How the query planner uses statistics: https://www.postgresql.org/docs/current/planner-stats-details.html
Hacking the PG statistics table: https://www.crunchydata.com/blog/hacking-the-postgres-statistics-tables-for-faster-queries

Statistics
Cardinality/Selectivity
Query **cost estimation** depends on the amount i.e. **cardinality** of data involved.

Selectivity is the proportion of the data i.e. **selectivity = cardinality / total**

Both the **selectivity of rows aka tuples** and the **selectivity of pages** are required depending on what is being estimated.
In general,

CPU cost	tuple selectivity, since process is per tuple
IO cost	page selectivity, since **storage** is in units of pages

Statistics collects values required to make **cardinality estimates**.
e.g.

reltuples	total cardinality of tuples
relpages	total cardinality of pages
most_common_vals	e.g. filter by equality WHERE x = 1
historgram_bounds	e.g. filter by inequality WHERE x <= 5

Statistics target
default_statistics_target
Defines how much **samples** to collect per stat.
e.g. if target=100, then **n_distinct** collects max 100 distinct values.
The default is **100**.

```
-- default stats
SET default_statistics_target TO 1000;

-- per column stats
ALTER TABLE mytable
  ALTER name SET STATISTICS 1000,
  ALTER value SET STATISTICS DEFAULT;
```

Single column statistics
Statistics of **pg_class**
i.e. stats per **table**.

relpages	number of page blocks on **disk** Used to determine **accuracy** of reltuples, and scale accordingly i.e. actual # tuples ~ actual # of pages / relpages * reltuples
reltuples	number of table **rows**

Statistics of **pg_statistics/pg_stats**
i.e. stats per **column**
pg_stats: https://www.postgresql.org/docs/current/view-pg-stats.html

N = **statistic target**

null_frac	0 ~ 1 fraction of NULLs
avg_width	average size (bytes) of the value
n_distinct	**number/proportion** of distinct values

n_distinct = **positive integer** if fixed number of distinct values and independent of table size
= number of distinct values

n_distinct = **[-1, 0)** if dependent on table size
= ratio of distinct values

e.g.
n_distinct = 11 11 distinct values
n_distinct = -0.4 (table size * 0.4) distinct values
n_distinct = -1 all distinct values

most_common_vals top N common values and frequencies
most_common_freqs

historgram_bounds N value histogram
Only for ordered values

correlation correlation between logical value **order** vs physical location (in storage)
Represents efficiency of reading
Only for ordered values

most_common_elems
most_common_freqs
elem_count_histogram

<u>* STATISTICS aka multivariate/extended statistics</u>
14.2 Statistics used by the planner: https://www.postgresql.org/docs/current/planner-stats.html
68.2 Multivariate statistics: https://www.postgresql.org/docs/current/multivariate-statistics-examples.html

Extended (aka multivariate) statistics
By default, statistics treated columns as **independent variables**
i.e. it does NOT account for **dependence** between columns.
Thus, the query planner's **cost estimate** may be inaccurate.
In turn, this may cause the query planner to select a less efficient **plan**.

e.g. a table of addresses

a **city** column is fully determined by the **address** column.
i.e. address has **exactly one** associated city
If

C = # cities
A = # addresses

by default, the database will assume there are at most **C * A** different combinations, which is inaccurate.
Thus, if the table is **grouped** by the cities and addresses, the query planner may prefer Group Aggregate instead of Hash Aggregate

Extended (multivariate) statistics define statistics that attempt to represent

dependencies between columns.
3 types:

1) **functional dependency**: whether column A is a **function** of B, and vice versa.

2) **n-distinct**: number of **distinct combinations** of columns (A, B).
e.g. useful when estimating the **number of groups** in GROUP BY.

3) **most common value (mvc)**: common combinations of columns (A, B).

CREATE STATISTICS: https://www.postgresql.org/docs/current/sql-createstatistics.html

```
CREATE STATISTICS [ [ IF NOT EXISTS ] statistics_name ]
    ON ( expression )
    FROM table_name

CREATE STATISTICS [ [ IF NOT EXISTS ] statistics_name ]
    [ ( statistics_kind [, ... ] ) ]
    ON { column_name | ( expression ) }, { column_name |
( expression ) } [, ...]
    FROM table_name

DROP STATISTICS [ IF EXISTS ] name [, ...] [ CASCADE | RESTRICT ]

statistics_kind = ndistinct | dependencies | mcv
```

*** Example: functional dependency**

case: **without** functional dependency

```
> select x a, x % 5 as b, x % 10 as c
into table t_dep
from generate_series(1, 10000) t(x);

> analyze verbose t_dep;
    -- ANALYZE to initialize stats i.e. normal single column stats

> explain select * from t_dep  where b = 1 and c = 6;
    -- IMPORTANT: currently only supports equality and IN

                        QUERY PLAN
-----------------------------------------------------------
 Seq Scan on t_dep  (cost=0.00..214.00 rows=200 width=12)
    -- BAD estimate
   Filter: ((b = 1) AND (c = 6))
    -- Thinks there are 5 * 10 = 50 possible values

    -- Hence 10000 / 50 = 200 rows
```

case: **with** functional dependency statistics

```
> create statistics st_dep (dependencies)
on a, b, c from t_dep;

> analyze verbose t_dep;
    -- ANALYZE to compute dependencies stats

> explain select * from t_dep  where b = 1 and c = 6;
    -- IMPORTANT: currently only supports equality and IN

                        QUERY PLAN
──────────────────────────────────────────────────────────
 Seq Scan on t_dep  (cost=0.00..214.00 rows=1000 width=12)
    -- GOOD esttimate
   Filter: ((b = 1) AND (c = 6))
    -- Understands that there are only 10 possible values of (b,
c)

    -- Hence 10000 / 10 = 1000 rows
```

See computed stats

```
> select stxname, stxkind, stxkeys, stxd.*
from pg_statistic_ext stx
inner join pg_statistic_ext_data stxd
on stx.oid = stxd.stxoid;

-- 1 => 2     a determines b
-- 1 => 3     a determines c
-- 3 => 2     c determines b
-- 1,2 => 3   a,b determines c
-- 1,3 => 2   a,c determines b
─[ RECORD
1 ]──┬──────────────────────────────────────────────────────────
─────────────────────────────────────────────────
stxname          | st_dep
stxkind          | {f}
stxkeys          | 1 2 3
stxoid           | 157109
stxdinherit      | f
stxdndistinct    | NULL
stxddependencies | {"1 => 2": 1.000000, "1 => 3": 1.000000, "3 =>
2": 1.000000, "1, 2 => 3": 1.000000, "1, 3 => 2": 1.000000}
stxdmcv          | NULL
stxdexpr         | NULL
```

*** Example: n distinct**

case: **without** statistiics

```
> select x a, x % 5 as b, x % 10 as c
        -- b is determined by c
into table t_ndis
from generate_series(1, 10000) t(x);

> analyze verbose t_ndis;
        -- ANALYSZE to initialize stats i.e.single column stats
> explain select b, c from t_ndis group by b, c;

                            QUERY PLAN
------------------------------------------------------------------------
 HashAggregate  (cost=214.00..214.50 rows=50 width=8)
        -- BAD estimate. Thinks there are 5 (b) * 10 (c) = 50
groups
   Group Key: b, c
   ->  Seq Scan on t_ndis  (cost=0.00..164.00 rows=10000 width=8)
```

case: with statistics

```
> create statistics st_ndis (ndistinct) on b, c from t_ndis;
> analyze verbose t_ndis;
        -- ANALYSZE to compute extended stats

-- thinks there are 10 distinct groups
explain select b, c from t_ndis group by b, c;

                            QUERY PLAN
------------------------------------------------------------------------
 HashAggregate  (cost=214.00..214.10 rows=10 width=8)
        -- GOOD estimate. Knows there are 10 (c) groups
   Group Key: b, c
   ->  Seq Scan on t_ndis  (cost=0.00..164.00 rows=10000 width=8)
```

See computed stats

```
> select stxname, stxkind, stxkeys, stxd.*
from pg_statistic_ext stx
inner join pg_statistic_ext_data stxd
on stx.oid = stxd.stxoid;

-[ RECORD
1 ]----+-----------------------------------------------------------------
-------------------------------------------------
stxname          | st_ndis
stxkind          | {d}
stxkeys          | 2 3
stxoid           | 157113
stxdinherit      | f
stxdndistinct    | {"2, 3": 10}
        -- 10 distinct groups: (b,c) = (0,0),(1,1),...(4,9)
stxddependencies | NULL
```

```
stxdmcv          | NULL
stxdexpr         | NULL
```

==========

==Vacuum/Analyze==

Routine vacuuming: https://www.postgresql.org/docs/current/routine-vacuuming.html

VACUUM https://www.postgresql.org/docs/current/sql-vacuum.html

```
VACUUM [ ( option [, ...] ) ] [ table_and_columns [, ...] ]

where option can be one of:

    FULL [ boolean ]
    FREEZE [ boolean ]
    VERBOSE [ boolean ]
    ANALYZE [ boolean ]
    DISABLE_PAGE_SKIPPING [ boolean ]
    SKIP_LOCKED [ boolean ]
    INDEX_CLEANUP { AUTO | ON | OFF }
    PROCESS_MAIN [ boolean ]
    PROCESS_TOAST [ boolean ]
    TRUNCATE [ boolean ]
    PARALLEL integer
    SKIP_DATABASE_STATS [ boolean ]
    ONLY_DATABASE_STATS [ boolean ]
    BUFFER_USAGE_LIMIT size

and table_and_columns is:

    table_name [ ( column_name [, ...] ) ]
```

ANALYZE: https://www.postgresql.org/docs/current/sql-analyze.html

```
ANALYZE [ ( option [, ...] ) ] [ table_and_columns [, ...] ]

where option can be one of:

    VERBOSE [ boolean ]
    SKIP_LOCKED [ boolean ]
    BUFFER_USAGE_LIMIT size

and table_and_columns is:

    table_name [ ( column_name [, ...] ) ]
```

Vacuum
In general, vacuuming has 2 purposes:
1) **cleanup** aka garbage collection:
In particular,
  a) storage cleanup + optimization: e.g. hard delete storage for deleted rows
  b) cleanup **transaction IDs**

2) **ANALYSIS** aka update statistics:
Used by **query planner**.

Vacuum configurations can be set at 2 levels:
1) **SYSTEM** aka configuration file:
i.e. either
 a) **system default**

 b) **configuration file** postgresql.conf
 Also includes **postgresql.auto.conf** set by **ALTER SYSTEM SET**.

2) per **TABLE**

e.g. system level configuraton
Show effective value from **pg_settings**

```
> select * from pg_settings where name = 'autovacuum_naptime';
-[ RECORD 1 ]---+----------------------------------------
name            | autovacuum_naptime
setting         | 60
unit            | s
category        | Autovacuum
short_desc      | Time to sleep between autovacuum runs.
extra_desc      | NULL
context         | sighup                                 --
sighup = effective value is system setting
vartype         | integer
source          | default
min_val         | 1
max_val         | 2147483
enumvals        | NULL
boot_val        | 60
reset_val       | 60
sourcefile      | NULL                                   --
null = not specified in config
sourceline      | NULL                                   --
i.e. is using system default
pending_restart | f
```

change with **ALTER SYSTEM SET/RESET**

```
> alter system set autovacuum_naptime='80s';

> select pg_reload_conf();
```

```
> select * from pg_settings where name = 'autovacuum_naptime';
-[ RECORD
1 ]---+-----------------------------------------------------------
name            | autovacuum_naptime
setting         | 80
unit            | s
category        | Autovacuum
short_desc      | Time to sleep between autovacuum runs.
extra_desc      | NULL
context         | sighup
        -- sighup = effective value is system setting
vartype         | integer
source          | configuration file
min_val         | 1
max_val         | 2147483
enumvals        | NULL
boot_val        | 60
reset_val       | 80
sourcefile      | /opt/homebrew/var/postgresql@17/
postgresql.auto.conf  -- source of system setting
sourceline      | 5
        -- i.e. non-default
pending_restart | f

> alter system reset autovacuum_naptime;
```

e.g. per table vacuum configuration

```
-- Table with custom vacuum option
> create table mytable (x int) with (autovacuum_enabled=off);

-- Check storage option
> select * from pg_class where relname = 'mytable' \gx

-[ RECORD 1 ]-------+-----------------------------
oid                 | 140589
relname             | mytable
relnamespace        | 2200
reltype             | 140591
reloftype           | 0
relowner            | 10
...

reloptions          | {autovacuum_enabled=off}                  --
option
relpartbound        | NULL
```

Autovacuum

autovacuum daemon/launcher/worker
https://www.postgresql.org/docs/current/routine-vacuuming.html#AUTOVACUUM
PostgreSQL has an **autovacuum daemon** to vacuum automatically.
By default, autovacuum is **enabled**.
Autovacuum executes when the **vacuum threshold** is surpassed:

vacuum threshold = base + scale * # tuples

base = **autovacuum_vacuum_threshold** configuration
scale = **autovacuum_vacuum_scale_factor** configuration
tuples = **reltuples** column of **pg_class** table
of rows in table

Likewise for **analysis threshold**.

<u>* usage: check last ANALYZE/VACUUM</u>
Check the stats table: https://www.postgresql.org/docs/current/monitoring-stats.html

```
SELECT * FROM pg_stat_all_tables

-- e.g.
relid               | 116133942
schemaname          | sch_slp
relname             | m_item_attribute_weekly_20250811
seq_scan            | 1279
seq_tup_read        | 5846040374
idx_scan            | 59229
idx_tup_fetch       | 22189741
n_tup_ins           | 52556297
n_tup_upd           | 70656995
n_tup_del           | 45183511
n_tup_hot_upd       | 124394
n_live_tup          | 6423677
n_dead_tup          | 52
n_mod_since_analyze | 52
n_ins_since_vacuum  | 0
last_vacuum         | NULL
last_autovacuum     | 2025-08-09 21:30:24.169061+00
last_analyze        | NULL
last_autoanalyze    | 2025-08-09 21:18:44.86887+00
vacuum_count        | 0
autovacuum_count    | 311
analyze_count       | 0
autoanalyze_count   | 35
...
```

==========

```
This
```

```
is
code
Monaco 14
```